SF

MGU BCA - 3RD SEM

Software Engineering

Complete Important Questions

By Aben Sojan

Exam - 10 Nov 2026

MODULE 1

Process and Agile

1

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q1-Q4 come from this module (process and agile). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2 (model paper essays: requirements elicitation, system testing). Old papers (May 2019 - Feb 2025) repeat SDLC questions verbatim: life-cycle activities 5x, waterfall 3x, Iterative-vs-RAD identical in Mar 2021 and Aug 2022. Attempt at least one question from each course outcome (CO1 = this module).

Every module, every important, repeated and sure-mark question for Software Engineering, each with a full-mark model answer.

1 Explain the evolving role of software.

IMPORTANT

Software is the set of programs and data that tell a computer what to do, and its role in society has changed completely over time.

In the early days, software was small and written by one person for one task, like a simple calculation program used by its own writer.

Then software became a product sold to many users, like operating systems and office tools.

Later, software moved to the internet as web applications that millions of people could use from anywhere.

Then came mobile apps, which put software in the pocket of almost every person on earth.

Today software runs banks, hospitals, transport, phones and the whole internet.

Cloud systems let companies use powerful software without owning any hardware.

Artificial intelligence systems now make decisions and create content like a human assistant.

So software changed from a small tool into the most important part of almost every business.

Example: a bank today is really a software company, because every account and payment is handled by software.

In short, software has grown from a helper program into the backbone of the modern world.

2 Explain the changing nature of software.

IMPORTANT

The nature of software means how it is built, who uses it and how it behaves, and this has changed a lot over time.

Old software was built for a single user and ran on one machine.

Modern software serves millions of users at the same time over the internet.

Old software was updated rarely, maybe once a year with a new version.

Modern software is updated continuously, sometimes many times a day.

Old software was a support tool that helped people do other work.

Today software is often the main product of the company itself, like a food delivery app or a video streaming service.

Modern software connects to other software through the internet and shares data constantly.

It must handle security, speed and huge amounts of data, which old software never faced.

Example: a simple calculator app of the past only added numbers, but a modern app like Google Pay moves real money safely for crores of users.

So the nature of software has changed from simple standalone tools to large, connected, always-changing systems.

3 Explain the importance of software engineering.

IMPORTANT

Software engineering is important because modern software is too large and complex to build by trial and error.

It gives a disciplined, step-by-step method for delivering software on time and within budget.

Without it, projects suffer missed deadlines, crossed budgets and poor quality.

It applies engineering ideas like planning, measurement and quality checks to software work.

It helps teams handle changing requirements without falling into chaos.

It reduces cost because finding and fixing an error early is far cheaper than fixing it after delivery.

It improves quality through reviews, testing and clearly defined processes.

It makes maintenance easier, since well-documented software can be changed by people who did not write it.

Example: a hospital management system built with proper requirements and testing will not lose patient records, while one built casually might.

In short, software engineering turns software building from risky guesswork into a controlled,

repeatable activity.

4 Describe the essential attributes of good software.

IMPORTANT

Good software has a set of essential attributes that make it useful and trustworthy.

Maintainability: it should be easy to change and fix when needs change.

Reliability: it should work correctly without frequent failures.

Efficiency: it should use time and memory wisely instead of wasting resources.

Usability: it should be easy to learn and pleasant to use.

Security: it should protect data from misuse and attacks.

Portability: it should run on different machines with little change.

These attributes often trade off against each other, so the team must balance them for the situation.

Example: a banking app is good software only if it is reliable with money, secure against fraud, and simple enough for anyone to use.

In short, good software is not just working code; it is maintainable, reliable, efficient, usable and secure.

5 Distinguish between generic software products and customized products.

IMPORTANT

Generic products are built for a wide market of customers; customized products are built for one specific customer.

For generic products, the requirements are decided by the developers through market study, as with Microsoft Word.

For customized products, the requirements are given by the customer under a contract, as with a billing system for one hospital.

Generic products are sold to many buyers; customized products are delivered to a single buyer.

Testing a generic product must cover many kinds of users and environments.

Testing a customized product focuses on the one client's workflows.

Generic products earn through volume sales; customized products earn through the development contract and maintenance.

Example: Tally is a generic accounting product, while the railway reservation system is a customized product built for Indian Railways.

In short, generic means made for the market, customized means made for one customer.

6 Explain layered technology in software engineering.

IMPORTANT

Software engineering is built in layers, like the floors of a building, where each layer rests on the one below it.

The top layer is the process layer, which defines the overall framework of activities used to build software.

The middle layer is the methods layer, which gives the technical how-to for each activity, like how to design or test.

The bottom layer is the tools layer, which gives automated support for the work, like editors, compilers and testing tools.

Beneath all three layers sits the quality focus, which supports everything and makes sure quality is checked at every step.

The layers depend on each other: a good process is useless without proper methods, and methods are slow without tools.

Example: for a project, the process could be Scrum, the method could be use-case modeling for design, and the tool could be Git for version control.

If any layer is weak, the whole building shakes, so all layers must be strong together.

In short, layered technology organises software engineering into process, methods and tools, all standing on quality.

7 What are the three layers of software engineering technology?

IMPORTANT

The three layers are process, methods and tools, supported by a quality focus at the base.

The process layer is the top layer; it defines the framework of activities, like communication, planning, modeling, construction and deployment.

It organises all the work in the correct order from start to finish.

The methods layer is the middle layer; it gives the technical how-to for each activity, like how to do analysis, design, coding or testing.

Example: writing UML diagrams is a design method.

The tools layer is the bottom working layer; it gives automated support, like IDEs, compilers, debuggers and test runners.

Example: Visual Studio Code and Git are tools.

The quality focus is not a fourth layer but the foundation that supports all three, keeping quality checks active everywhere.

Together, process tells what to do and when, methods tell how to do it, and tools help do it faster and with fewer errors.

8 **What are the life-cycle phases? Explain the life-cycle activities of software development. [Asked: May 2019, Mar 2021, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

The software life cycle is the series of phases a product passes through from birth to retirement.

The main life-cycle phases are requirements analysis, design, coding, testing, deployment and maintenance.

Requirements analysis discovers what the customer actually needs.

Design plans how the software will be built: its structure, modules and screens.

Coding is the actual writing of the program from the design.

Testing checks that the software works correctly and meets the requirements.

Deployment installs the software for real users.

Maintenance fixes problems and adds new features after delivery.

Each phase ends with a review, so errors are caught before they flow into the next phase.

Example: for a railway ticket app, the team first studies booking needs, then designs screens, codes the app, tests booking flows, releases it, and later adds seat-map features.

In short, the life-cycle phases carry software from an idea to a working product and keep it alive afterwards.

9 **Define software process. [Asked: Aug 2022, Feb 2025]**

REPEATED

A software process is the set of related activities that leads to the production of a software product.

It defines which activities are done, in what order, and who performs them.

Typical activities include specification, design and implementation, validation, and evolution.

Every software project follows some process, whether it is written down or followed informally.

A defined process makes the work predictable, repeatable and controllable.

Without a process, projects depend on individual heroics and frequently miss deadlines.

Different projects need different processes: waterfall suits stable requirements, agile suits changing ones.

The choice depends on project size, team skill, risk and customer involvement.

Example: a college building an admission portal with fixed requirements can safely use the waterfall process.

In short, the software process is the roadmap a team follows from an idea to working software.

10 What is a process framework? Explain its activities.

IMPORTANT

A process framework is the common skeleton of activities that every software project follows, whatever model it uses.

It defines five framework activities.

Communication: talk to the customer and stakeholders to understand what they need.

Planning: decide the work to do, the schedule, the cost and the risks.

Modeling: create models of what to build and how it will work, like analysis and design models.

Construction: write the code and test it to find and fix errors.

Deployment: deliver the software to the customer and collect feedback for improvement.

Alongside these, umbrella activities run through the whole project.

Umbrella activities include project tracking, risk management, quality assurance and configuration management.

Example: quality assurance is not a separate stage; it checks the work done in every activity, from planning to deployment.

Different process models like waterfall and agile adapt this same framework in their own way.

11 Distinguish between framework activities, actions and tasks.

IMPORTANT

Framework activities, actions and tasks are three levels of work in a process framework, from big to small.

Framework activities are the main stages of a project, like communication, planning, modeling, construction and deployment.

They are the same for every project and give the big picture.

Actions are smaller steps inside one framework activity.

Example: inside the planning activity, actions include creating a project schedule and doing risk analysis.

Tasks are the smallest units of work, usually done by one person in a short time.

Example: writing the login screen test cases is a single task inside the testing action.

So one framework activity contains many actions, and one action contains many tasks.

This division helps managers plan and track work at the right level of detail.

12 Differentiate product and process.

IMPORTANT

Product is what is built; process is how it is built.

The product is the final software delivered to the customer, like an app or a website.

The process is the set of activities used to create that product, like design, coding and testing.

Product quality is judged by the user; process quality is judged by how smoothly the work flows.

A good process usually leads to a good product, but a good product can sometimes emerge from a weak process by luck.

Products are measured by features, speed and reliability; processes are measured by cost, time and repeatability.

Organisations improve the process so that every future product comes out better.

Example: the banking app on your phone is the product; the Scrum sprints the team used to build it are the process.

In short, the product is the destination and the process is the road taken to reach it.

13 What are the drawbacks of the waterfall model? [Asked: Mar 2021, Feb 2025]

REPEATED

The biggest drawback of the waterfall model is that real projects rarely have fixed requirements at the start.

Customers often change their minds, and in waterfall any change after a stage is done is very expensive.

The customer sees working software only at the very end of the project.

If the final product is wrong, months of work may be wasted.

There is no early feedback, so misunderstandings are found very late.

The model does not handle risks well during development, because testing and integration happen only near the end.

It also produces a lot of documents, which slows down small and fast-moving projects.

Example: if a shopping app is built in waterfall and the customer wanted a different payment option, the team must redo design and code from scratch.

So waterfall is risky whenever requirements are unclear or likely to change.

14 **How do you select an appropriate life-cycle model for a project? Illustrate with examples. [Asked: May 2019, Feb 2025]**

REPEATED

The life-cycle model is selected by matching the project's nature with each model's strengths.

First, check requirement stability: stable requirements suit waterfall; changing ones suit agile or spiral.

Second, check project size and risk: large, risky projects suit the spiral model, which handles risk in every loop.

Third, check customer involvement: if the customer can review often, prototyping or agile works well.

Fourth, check deadlines: tight deadlines with clear requirements suit RAD or incremental delivery.

Fifth, check team skill: agile needs experienced, self-managing teams.

Sixth, check how critical failure is: life-critical systems need the heavy verification of plan-driven models.

Example: a fixed-requirement exam-result portal suits waterfall; a startup's experimental app suits Scrum; a satellite control system suits spiral because failure is not an option.

In short, there is no best model for all projects; the right model is the one that fits the project's requirements, risks and people.

15 **Explain the waterfall model with its stages.**

IMPORTANT

The waterfall model is the oldest and simplest process model, where work flows down through fixed stages like water falling over steps.

The first stage is requirements: gather and document everything the customer needs.

The second stage is design: plan the software structure based on the requirements.

The third stage is implementation: write the actual code.

The fourth stage is testing: check the software for errors and fix them.

The fifth stage is deployment: deliver the finished software to the customer.

The sixth stage is maintenance: fix problems and make small improvements after delivery.

Each stage must be fully finished before the next one starts.

Going back to an earlier stage is difficult and costly, so requirements must be clear from the beginning.

Example: building software for an ATM machine works well, because the rules for cash

withdrawal are fixed and well known.

In short, waterfall is simple and easy to manage, but only fits projects where requirements will not change.

16 Compare waterfall model with agile development.

IMPORTANT

Waterfall and agile are opposite styles of building software.

Waterfall moves in fixed stages in one direction: requirements, design, coding, testing and deployment.

Agile works in small cycles called iterations or sprints, repeating planning, building and testing many times.

In waterfall, change is costly because a finished stage must be redone.

In agile, change is welcome even late in the project, because only small pieces are rebuilt.

In waterfall the customer sees working software only at the end.

In agile the customer gets working software every few weeks and can give feedback early.

Waterfall needs heavy documentation; agile keeps documentation light and focuses on working software.

Example: a government tax system with fixed rules suits waterfall, while a new food delivery app with changing ideas suits agile.

In short, waterfall fits stable requirements and agile fits changing requirements.

17 When is the waterfall model suitable?

IMPORTANT

The waterfall model is suitable when the requirements are well understood and unlikely to change.

It works best for small projects with a fixed scope and clear goals.

It suits projects that need strict documentation, like banking or medical systems.

It is good when the technology is familiar and the team knows exactly how to build it.

It is also useful when the customer cannot give regular feedback, since the model does not depend on it.

In such cases the simplicity of waterfall becomes a strength, because the plan can be followed step by step.

Example: software for a simple billing system with fixed tax rules is a perfect fit.

The opposite case is a startup building a new social app, which should avoid waterfall because

its ideas will change often.

So waterfall is suitable when the road is clear, fixed and well mapped in advance.

18 Write a note on project risks in the selection of a life-cycle model.

IMPORTANT

Choosing the wrong life-cycle model is itself a major project risk.

If requirements are unstable and waterfall is chosen, late changes become extremely expensive.

If the team is inexperienced and agile is chosen without guidance, sprints can drift without direction.

Technical risks, like unproven technology, need a model with early prototypes or risk loops, such as the spiral model.

Schedule risks need models that deliver early usable versions, like incremental or RAD.

Customer-related risks, like an unavailable customer, rule out models needing constant feedback, such as XP.

The selection should therefore start with a risk assessment, not with habit or fashion.

Example: picking waterfall for a hospital system whose rules keep changing risks delivering a system nobody wants.

In short, model selection must weigh requirement, technical, schedule and people risks before the project starts.

19 What are the advantages of the incremental model? [Asked: May 2019, Feb 2025]

REPEATED

The first advantage of the incremental model is early delivery: the customer gets working software quickly instead of waiting for the whole project.

Early delivery means the customer can start getting value and testing the product sooner.

The second advantage is feedback: comments on early increments improve the later ones.

The third advantage is lower risk, because problems are found in small pieces instead of one big failure at the end.

It is also easier to handle changes, since new requirements can be added to a future increment.

Testing is simpler because each small increment is tested alone before joining the system.

The team and customer both stay motivated, because progress is visible after every increment.

Example: in a railway booking site, the ticket booking increment can go live while the refund increment is still being built.

So the incremental model gives speed, flexibility and safety together.

20 Explain the incremental process model.

IMPORTANT

The incremental model builds software in small pieces called increments instead of all at once.

The first increment is a basic working version with only the core features.

Each later increment adds more features until the full system is complete.

Every increment goes through the same mini-process: requirements, design, coding and testing.

The customer gets usable software very early and can start using it.

Feedback on early increments is used to improve the later ones.

If one increment has a problem, it does not break the whole project, because the work is in small pieces.

Example: a college portal can first release login and notices, then add fee payment in the second increment, and results in the third.

In short, incremental delivery gives early value, early feedback and lower risk.

21 Compare the Iterative model with the RAD model. [Asked: Mar 2021, Aug 2022]

REPEATED

Both build software in repeated cycles, but they differ in goal and method.

Iterative model: each iteration adds new features to a growing product; the focus is gradual refinement.

RAD: each cycle aims to deliver a complete working system fast, using reusable components.

The iterative model suits projects where requirements are not fully known at the start.

RAD suits projects where requirements are well understood and speed is the priority.

The iterative model manages risk by reviewing the growing product after every iteration.

RAD manages time by fixing short cycles and reusing proven parts.

The iterative model needs strong customer feedback between iterations.

RAD needs highly skilled developers and a library of reusable components.

Example: a new social media app fits the iterative model, while a standard company payroll system fits RAD.

In short, the iterative model grows the product step by step, while RAD races to a finished system using ready parts.

22 What are evolutionary process models? Explain any two.

IMPORTANT

Evolutionary models accept that software grows and changes over time instead of being fixed at the start.

They build the software in versions that improve with customer feedback.

The first example is prototyping.

In prototyping, a quick rough version of the software is built and shown to the customer.

This helps everyone understand unclear requirements before the real system is built.

The prototype is usually thrown away, and the final system is built after the requirements are clear.

The second example is the spiral model.

The spiral model works in repeated loops; each loop has planning, risk analysis, engineering and customer review.

It is risk-driven, so the most dangerous and uncertain parts are handled first.

Example: for a new online exam system, a prototype can show how the exam screen should look, and the spiral model can manage the risk of handling thousands of students at once.

Both models use feedback and repetition to grow the software safely.

23 What is prototyping? When is it used?

IMPORTANT

A prototype is a quick rough model of the software that is shown to the customer before the real system is built.

It looks and partly works like the final product, but it is not the final product.

Prototyping is used when the customer cannot explain clearly what they want.

It is used when requirements are confusing or new technology is involved.

It is also used when the user interface is important and must be approved early.

The customer tries the prototype, gives feedback, and the requirements become clear.

The prototype is usually thrown away after this, and the real system is built properly.

This saves money, because fixing a misunderstanding in a rough prototype is far cheaper than fixing finished code.

Example: for a hospital management app, a prototype can show the patient registration screen so doctors can say what they actually need.

In short, prototyping turns unclear ideas into clear requirements.

24 Explain the spiral model and its advantages.

IMPORTANT

The spiral model was proposed by Barry Boehm and works in repeated loops called spirals.

Each loop passes through four parts.

Planning: decide the goals and limits for this loop.

Risk analysis: find the biggest risks and decide how to reduce them; this is the heart of the spiral model.

Engineering: build and test a part of the software.

Customer review: show the result to the customer and plan the next loop.

The main advantage is risk handling: high-risk parts are faced first, so big failures are avoided.

Another advantage is early customer feedback in every loop.

It suits large, complex and high-risk projects very well.

The drawback is that it needs expert risk analysis, which makes it costly for small projects.

Example: software for a satellite launch would use the spiral model because the risk of failure is huge.

So the spiral model trades extra planning effort for safety on big risky projects.

25 What is RAD? Explain the RAD model.

IMPORTANT

RAD stands for Rapid Application Development, a model that builds software very fast using reusable components.

It works in short cycles of about 60 to 90 days, delivering a working system quickly.

RAD has four phases: business modeling, data modeling, process modeling, and application generation and testing.

Business modeling studies what information drives the business.

Data modeling organises that information into data objects.

Process modeling describes how the data is used to complete business tasks.

Application generation builds the system using ready-made components and automated tools, then tests it.

RAD needs a highly skilled team and works best when requirements are well understood.

Example: building a standard inventory system for a shop chain suits RAD because the requirements are clear and components can be reused.

In short, RAD trades detailed upfront planning for speed by reusing components in short, sharp cycles.

26 If requirements are frequently changing, which life-cycle model should be selected? Justify.

IMPORTANT

When requirements change frequently, an agile or evolutionary model should be selected.

The waterfall model fails here because it freezes requirements early, making later changes very costly.

Agile models like Scrum welcome changing requirements even late in development.

They work in short sprints, so new or changed requirements simply enter the next sprint backlog.

Evolutionary models like prototyping also fit, because the customer sees working versions early and corrects course.

The spiral model fits too, since each loop re-examines risks, including the risk of requirement change.

The key test is: does the model expect change or punish it?

Example: a startup building a food-delivery app with shifting ideas should use Scrum, adding new features sprint by sprint.

In short, pick a model that expects change: agile or evolutionary, never a rigid plan-driven one.

27 Explain the Unified Process with its phases.

IMPORTANT

The Unified Process, or UP, is an iterative process model that is driven by use cases.

Use cases describe how users interact with the system, and they guide all the work.

UP is also architecture-focused, meaning the basic structure of the system is fixed early.

It has four phases.

Inception: understand the idea, define the scope, and get approval to continue.

Elaboration: design the architecture, remove the major risks, and make a detailed plan.

Construction: build the software in iterations until it is complete.

Transition: deliver the software to users, train them and fix the first problems.

Work is organised into workflows like requirements, analysis, design, implementation and test that run across the phases.

UP suits large object-oriented projects that need a strong and stable architecture.

Example: a bank building a new core banking system would use UP to keep the architecture solid from the start.

28 What happens in each phase of the Unified Process?

IMPORTANT

Each phase of the Unified Process has a clear goal and a clear result.

Inception: the team defines the project scope, lists the main use cases, checks if the project is possible, and gets approval.

Its result is a yes-or-no decision to continue.

Elaboration: the team designs the system architecture, builds a working skeleton, removes major technical risks and makes the full plan.

Its result is a stable architecture everyone agrees on.

Construction: the team builds the software in iterations, adding features and testing them each time.

Its result is a complete and tested system.

Transition: the team delivers the software to users, trains them, fixes early problems and hands over documentation.

Its result is a live system running with real users.

Example: in a railway reservation project, inception approves the idea, elaboration designs how booking works, construction builds it, and transition puts it in stations.

So the four phases move the project safely from idea to live use.

29 Distinguish between plan-driven and agile development. [Asked: Model paper]

REPEATED

Plan-driven and agile are two opposite ways of managing software projects.

In plan-driven development, the full plan is made first: requirements, design and schedule are fixed before coding.

In agile, planning adapts as work continues; only the near future is planned in detail.

In plan-driven work, change is costly because the plan must be redone.

In agile, change is welcome at any time because work is done in small pieces.

Plan-driven projects produce heavy documentation at every stage.

Agile projects keep documentation light and focus on working software.

The customer is involved mainly at the start and end in plan-driven work, but daily in agile work.

Plan-driven suits fixed, critical systems like flight control software.

Agile suits fast-changing products like mobile apps and websites.

Example: building a bridge follows a plan-driven style, while building a new food delivery app follows an agile style.

In short, plan-driven trusts the plan, agile trusts the team and the feedback.

30 Explain the principles of agility.

IMPORTANT

Agility is the ability of a team to respond quickly and correctly when requirements change.

It is guided by several principles from the Agile Manifesto.

Satisfy the customer through early and continuous delivery of useful software.

Welcome changing requirements, even late in development, because change gives the customer an advantage.

Deliver working software frequently, in short periods of a few weeks.

Business people and developers must work together daily throughout the project.

Build projects around motivated people, give them support, and trust them.

The most efficient communication is face-to-face conversation.

Working software is the main measure of progress.

Simplicity is essential: do the maximum work with the minimum effort.

Teams regularly reflect on how to improve and adjust their behaviour.

Example: a startup that changes its app design after user feedback within one week is showing agility in action.

In short, agility means fast delivery, easy change and close teamwork.

31 State and explain the four values of the Agile Manifesto.

IMPORTANT

The Agile Manifesto states four values that guide all agile work.

First: individuals and interactions over processes and tools.

This means talented people talking to each other matter more than rigid rules and fancy software.

Second: working software over comprehensive documentation.

This means a running product matters more than thick documents nobody reads.

Third: customer collaboration over contract negotiation.

This means working with the customer daily is better than fighting over a fixed contract.

Fourth: responding to change over following a plan.

This means adapting to new needs is better than blindly following an old plan.

Note that the items on the right still have value, but the items on the left are valued more.

Example: a team that quickly rebuilds a feature after customer feedback is following the fourth value.

These four values are the heart of every agile method like Scrum and XP.

32 Explain Extreme Programming and its practices. [Asked: Model paper]

REPEATED

Extreme Programming, or XP, is an agile method that takes good practices to an extreme level.

It uses very short development cycles with constant feedback from the customer.

Its key practices are the following.

Pair programming: two developers work on one machine; one writes code while the other reviews it live.

Test-driven development: write the test first, then write the code that passes the test.

Continuous integration: merge and test all code many times a day so errors are found early.

Simple design: keep the design as simple as possible and improve it later.

Refactoring: regularly clean and improve the code without changing what it does.

The customer is always available on site to answer questions quickly.

XP suits small teams building software with changing requirements.

Example: a team of five building a college event app can use pair programming and daily integration to stay fast and correct.

So XP delivers quality software by pushing teamwork, testing and simplicity to the extreme.

33 Compare Extreme Programming with Scrum. [Asked: Model paper]

REPEATED

XP and Scrum are both agile methods, but they focus on different things.

XP focuses on engineering practices: how the code is actually written.

Its core is pair programming, test-driven development, continuous integration and simple design.

Scrum focuses on project management: how the work is organised.

Its core is roles, sprints and meetings like planning, standup, review and retrospective.

XP says little about who manages the project; Scrum says little about how to write code.

XP suits small technical teams that need coding discipline.

Scrum suits teams of any size that need clear planning and tracking.

Because they cover different gaps, many teams use both together.

Example: a team can run Scrum sprints for planning and use XP's pair programming and test-first coding inside each sprint.

In short, Scrum manages the work and XP builds the work, so they fit together well.

34 Explain Scrum with its roles and meetings.

IMPORTANT

Scrum is the most popular agile method; it organises work into fixed-length cycles called sprints.

A sprint is usually 2 to 4 weeks long and ends with working software.

Scrum has three roles.

Product Owner: decides what to build and sets the priority of features.

Scrum Master: removes problems that block the team and protects the Scrum process.

Development Team: the developers who build the software; they organise their own work.

Scrum has four main meetings.

Sprint planning: choose the work for the coming sprint.

Daily standup: a 15-minute meeting on progress and blockers.

Sprint review: demo the finished work to stakeholders.

Retrospective: discuss what went well and what to improve.

Example: a team building a cab booking app can deliver login in sprint one, booking in sprint two, and payment in sprint three.

Scrum keeps the team focused, visible and improving every sprint.

35 Explain the Scrum meetings in detail.

IMPORTANT

Scrum uses four meetings to keep the sprint on track.

Sprint planning happens at the start of the sprint.

The team picks items from the product backlog and decides what can be finished in this sprint.

The result is the sprint backlog, the team's promise for the sprint.

Daily standup is a short 15-minute meeting held every day at the same time.

Each member answers three questions: what did I do yesterday, what will I do today, and what is blocking me.

It is not a status report to a boss; it is the team helping each other.

Sprint review happens at the end of the sprint.

The team demos the working software to the product owner and stakeholders and collects

feedback.

Retrospective also happens at the end, after the review.

The team honestly discusses what went well, what went wrong and what to change next sprint.

Example: a retrospective may reveal that testing was rushed, so the next sprint adds more test time.

These four meetings give Scrum its rhythm of plan, work, show and improve.

36 What are product backlog, sprint backlog and increment in Scrum?

IMPORTANT

These three are the main artefacts, or work products, of Scrum.

The product backlog is the full ordered list of everything the product may ever need.

It contains features, fixes and improvements, with the most important items on top.

The Product Owner owns it and keeps it updated.

The sprint backlog is the small list of items chosen for the current sprint.

It is created in sprint planning and belongs to the Development Team.

Nobody changes it during the sprint, so the team can focus.

The increment is the working software produced at the end of the sprint.

It must be potentially shippable, meaning it could be given to the customer.

Each sprint's increment adds to all the previous ones, growing the product step by step.

Example: for a food app, the product backlog has 50 features, the sprint backlog has 5 for this sprint, and the increment is the working login and menu screens.

So the product backlog is the whole dream, the sprint backlog is this sprint's promise, and the increment is the finished result.

37 Explain the agile tool set with examples. [Asked: Model paper]

REPEATED

The agile tool set is the collection of tools that help a team work in an agile style.

Task boards track work visually: tools like Trello and Jira show tasks moving from to-do to doing to done.

Version control tools like Git let many developers work on the same code safely.

Every change is recorded, so mistakes can be undone and work can be merged.

Continuous integration tools like Jenkins build and test the code automatically on every change.

This finds errors within minutes instead of weeks.

Communication tools like Slack and Zoom keep the team in daily contact.

They support daily standups and quick discussions, especially for remote teams.

Test automation tools run the test suite fast after every change.

Example: a team pushes code to Git, Jenkins auto-builds and auto-tests it, and the board card moves to done automatically.

Together these tools make agile speed and quality possible.

38 **What is a Kanban board? How does an agile team use it?**

IMPORTANT

A Kanban board is a visual board that shows the status of every task in the project.

It has columns like To Do, Doing and Done.

Each task is written on a card placed in the correct column.

As work progresses, team members move their cards from left to right.

The board can be physical, like a whiteboard with sticky notes, or digital, like Trello or Jira.

Everyone can see the project status at a glance without calling a meeting.

If too many cards pile up in the Doing column, the team knows work is stuck and helps out.

This keeps the work flowing smoothly and stops people from starting too many tasks at once.

Example: a student team building a website puts design, coding and testing cards on the board and moves them daily.

So a Kanban board makes work visible, simple and easy to manage.

MODULE 2

Requirements, Risk and Planning

2

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q5-Q7 come from this module (requirements and planning). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2; the model paper essay is requirements elicitation and analysis - the single most likely 15-marker. COCOMO appeared 4 times (May 2019, Oct 2021, Aug 2022, Feb 2025) - sure mark; function points 2x. RMMM, risk tables and agile planning have zero past-paper precedent: learn them from the syllabus as IMPORTANT, not as repeats.

1 What is requirements engineering? Why is it important?

IMPORTANT

Requirements engineering is the process of discovering, documenting and managing what the customer wants the software to do.

It starts by talking to all stakeholders like customers, users and managers to collect their needs.

These needs are written clearly in a requirements document so nothing is missed or misunderstood.

Requirements are then checked and agreed upon before any design or coding begins.

It is the first and most important stage of software development.

Studies show that fixing a wrong requirement after coding costs many times more than fixing it at the start.

Many project failures happen not because of bad coding but because the software was built for the wrong needs.

For example, if a bank app misses the fund transfer feature, the whole product fails no matter how good the code is.

So good requirements engineering saves time, money and customer trust.

2 Distinguish between functional and non-functional requirements with examples.

IMPORTANT

Functional requirements describe what the system must do, the services it provides.

For example, an online shop must allow user registration, add to cart and online payment.

Non-functional requirements describe how well the system must do it, the quality limits.

For example, the payment page must load within 2 seconds and use secure encryption.

Functional requirements are tested by checking if the feature works or not.

Non-functional requirements are tested by measuring speed, security, usability and reliability.

An ATM example: dispensing cash is functional, doing it within 30 seconds is non-functional.

Both types are equally important because a feature that works too slowly is as bad as a missing feature.

In the exam, always pair each functional example with a matching non-functional limit to get full marks.

3 **Give four examples of non-functional requirements.**

IMPORTANT

Performance: the system must display search results within 2 seconds.

Security: all user passwords must be stored in encrypted form and never as plain text.

Usability: a new user must be able to learn and use the app without help within 10 minutes.

Reliability: the system must not fail more than once in a month of normal use.

Each of these is written with a clear number so it can be tested.

A vague statement like the system should be fast is not a proper non-functional requirement.

For example, in a railway booking site, non-functional needs decide whether crores of users can book tickets at once.

Good non-functional requirements turn customer wishes into measurable targets for the team.

4 **What is a requirements document? Explain its contents. [Asked: Model paper]**

REPEATED

The requirements document is the agreed written record of everything the software must do.

It acts as the contract between the customer and the development team.

Its contents begin with the introduction: the purpose of the system and who will use it.

Next comes the system scope: what is included and what is not included.

Then the functional requirements: the full list of services the system must provide.

Then the non-functional requirements: speed, security, usability and reliability targets.

It also lists constraints like budget, deadlines and technology limits, plus the assumptions made.

Both customer and developers sign off on this document before building starts.

Any change later must go through a formal change process because this document is the baseline.

5 What is a software specification? What makes it good?

IMPORTANT

A software specification describes each requirement precisely enough to build and test it.

A good specification must be correct: it says exactly what the customer wants.

It must be unambiguous: each line has only one possible meaning.

It must be complete: nothing important is left out.

It must be verifiable: there is a way to test whether it is done.

For example, the system shall respond within 2 seconds is verifiable, while fast response is not.

It must be consistent: no two requirements should contradict each other.

Bad specifications cause arguments later, so writing them well saves the whole project.

6 What do you mean by scope in SRS?

IMPORTANT

Scope in an SRS, the Software Requirements Specification, defines the boundaries of what the software will and will not do.

It states the main goals and the features included in the project.

It also clearly lists what is excluded, so expectations stay realistic.

A clear scope prevents scope creep, where extra demands keep entering during development.

It helps in estimating cost and time, because the team knows exactly what must be built.

It gives a basis for accepting or rejecting change requests later.

Both the customer and the developers sign off on the scope before work begins.

Example: for a library app, the scope includes book issue and return but excludes online fee payment, which is stated as out of scope.

In short, scope draws the fence around the project so everyone agrees on what is inside.

7 **Explain requirements elicitation and analysis. [Asked: Model paper]**

SURE MARK

Requirements elicitation and analysis is the heart of requirements engineering, where the team discovers and understands what to build.

Elicitation means gathering requirements from stakeholders through interviews, questionnaires, observation and workshops.

FAST, the Facilitated Application Specification Technique, brings customers and developers together in a guided joint session.

Prototyping is also used: showing a rough version helps customers say what they really want.

Analysis then studies the gathered requirements for conflicts, gaps, overlaps and feasibility.

Conflicting requirements are negotiated and resolved with the stakeholders.

Unclear requirements are clarified before they are written into the requirements document.

The output is a clear, agreed and feasible set of requirements.

Example: for a college bus-tracking app, the team interviews students and drivers, builds a screen mockup, finds that live GPS conflicts with battery limits, and negotiates a 30-second update interval.

In short, elicitation collects the wishes and analysis turns them into a consistent, buildable plan.

8 **Why are requirements difficult to uncover? Explain. [Asked: Mar 2021, Feb 2025]**

REPEATED

Requirements are difficult to uncover because customers often cannot state clearly what they need.

Users know their daily work but not how to express it as software features.

Different stakeholders give conflicting requirements that must be reconciled.

Requirements keep changing as customers see early versions and get new ideas.

Some requirements stay hidden because users assume certain things are obvious and never mention them.

Technical limits may make some wishes impossible, which only emerges during analysis.

The problem grows in large systems where hundreds of stakeholders are involved.

Skilled analysts use prototypes, observation and careful questioning to dig out the real needs.

Example: a shop owner says make billing fast but cannot define fast; the analyst discovers it means under 10 seconds per bill.

In short, elicitation is hard because human needs are vague, conflicting and moving, and the analyst must dig them out carefully.

9 Explain the requirements engineering processes.

IMPORTANT

Requirements engineering has four main processes that run in a loop.

Elicitation and analysis: gather the needs from stakeholders and understand them.

Specification: write the needs down clearly in the requirements document.

Validation: check that the written requirements are correct, complete and realistic.

Management: control changes to requirements as the project continues.

They are not done once; they repeat until everyone agrees and requirements are stable.

For example, a customer may add a new feature mid-project, which goes through all four steps again.

Skipping any of these steps leads to missing or wrong requirements and project failure.

10 Explain the techniques of requirements elicitation.

IMPORTANT

Interviews: meet stakeholders one by one and ask open questions about their needs.

Questionnaires: send written questions to collect answers from many users quickly and cheaply.

Observation: watch users doing their present work to find needs they forgot to mention.

Workshops: bring all stakeholders into one room to discuss and agree fast.

Document study: read existing forms, reports and manuals to understand current processes.

Each technique suits a different situation, so teams usually combine two or three of them.

For example, a hospital system team may interview doctors, observe nurses and hold a workshop with the admin staff.

Using only one technique often misses hidden needs, so mixing them gives complete requirements.

11 What is requirements analysis? Why is it needed?

IMPORTANT

Requirements analysis studies the gathered requirements to find conflicts, gaps and impossible demands.

Different stakeholders often ask for opposite things without realising it.

Example: one user wants instant reports while another wants heavy security checks on every report, and analysis resolves this conflict.

It also finds missing details, like what should happen when a payment fails.

Impossible demands are removed early, like a mobile app working with zero internet forever.

Tools like data flow diagrams and use case diagrams help model and check the requirements.

Without analysis, developers start coding on confused and conflicting requirements.

So analysis turns a raw pile of wishes into a clean, workable set of requirements.

12 What is requirements validation? Explain its techniques.

IMPORTANT

Requirements validation checks that the requirements are correct, complete and realistic.

It asks two questions: did we capture what the customer truly wants, and can we actually build it.

Technique one is reviews: stakeholders read the document together and point out errors.

Technique two is prototyping: build a quick demo so the customer can see and confirm.

Technique three is test-case writing: writing tests early exposes unclear or untestable requirements.

Catching an error during validation is far cheaper than catching it after coding is done.

For example, a review may reveal that a report the customer needs was never written down.

Validation is the quality gate that stops wrong requirements from entering design.

13 What is requirements management?

IMPORTANT

Requirements management controls changes to requirements during the project.

Requirements always change because customers understand their needs better over time.

Each requirement is tracked with a number, its version and its current status.

Every change request is analysed for cost, time and impact before it is approved.

Approved changes are updated in the document so everyone works on the latest version.

For example, adding fingerprint login mid-project needs approval and a schedule change.

Without management, uncontrolled changes cause confusion, delays and budget overruns.

Good management keeps requirements stable enough to build while still allowing useful changes.

14 What is negotiation in requirements engineering?

IMPORTANT

Negotiation is the process of resolving conflicts between different stakeholders' requirements.

Different users often want opposite things: managers want low cost while users want many features.

The analyst brings the parties together and helps them agree on priorities.

Less important requirements may be dropped or moved to a later release.

Win-win solutions are found where each side gives up something small.

Technical limits are explained honestly so stakeholders adjust their expectations.

The agreed decisions are recorded in the requirements document so nobody reopens settled points.

Example: students want bus tracking every 5 seconds but the transport office worries about data cost; they negotiate a 30-second interval.

In short, negotiation turns conflicting wishes into one agreed plan everyone can live with.

15 What is Facilitated Application Specification Technique (FAST)?

IMPORTANT

FAST is a team-based technique for gathering requirements in a guided joint session.

Customers and developers meet together instead of working through long separate interviews.

A neutral facilitator controls the meeting and keeps it focused on decisions.

Before the meeting, each participant prepares lists of what the system should do, what is already known, and what is unclear.

In the session the group discusses each list, removes duplicates and resolves conflicts on the spot.

A combined draft specification is prepared during or immediately after the meeting.

FAST works well when stakeholders are available and decisions need speed.

Example: for a canteen billing system, the owner, cashiers and developers sit together for a day and finalise the billing rules in one session.

In short, FAST speeds up elicitation by replacing slow one-by-one interviews with one focused team workshop.

16 What is software risk? Explain reactive vs proactive strategies.

IMPORTANT

Software risk is any uncertain event that can harm the project if it happens.

It can hurt the schedule, the cost, the quality or even the success of the product.

A reactive strategy deals with problems only after they happen, which is also called firefighting.

The team waits for trouble and then rushes to fix it, which is costly and stressful.

A proactive strategy identifies risks early and plans how to avoid or reduce them.

The team prepares backup plans before anything goes wrong.

Proactive is always better because fixing a problem late costs much more than preventing it.

For example, training two people on a critical module is proactive, while struggling after one quits is reactive.

17 Compare reactive and proactive risk strategies with an example.

IMPORTANT

Reactive strategy has no risk planning; the team panics when trouble arrives.

Problems cause sudden delays, overtime work and poor quality fixes.

Proactive strategy lists risks early and keeps backup plans ready.

When a risk occurs, the team calmly follows the prepared plan.

Example: if a key developer may quit, proactive means training a second developer now.

The reactive version is hiring urgently after the developer has already quit, delaying the project.

Proactive teams stay on schedule while reactive teams keep firefighting.

So successful projects always choose the proactive strategy.

18 Explain the types of software risks with examples.

IMPORTANT

Project risks threaten the schedule or the budget of the project.

Example: a key developer resigns, or the hardware delivery is delayed by a month.

Technical risks threaten the quality or performance of the software.

Example: the chosen database cannot handle the expected number of users.

Business risks threaten the success or viability of the product in the market.

Example: customers do not want the product, or a competitor releases a better one first.

Knowing the type of risk helps the team choose the right response.

A project risk needs a backup plan for people, while a technical risk needs testing or a new design.

19 What are known, predictable and unpredictable risks?

IMPORTANT

Known risks can be listed from experience before the project starts.

Example: delay in hardware delivery or unclear customer requirements.

Predictable risks are estimated from data of past projects.

Example: the staff turnover rate in the company tells us some developers may leave.

Unpredictable risks are surprises that no one could see coming.

Example: a sudden new government law changes the tax rules the software must follow.

Known and predictable risks can be planned for in advance.

Unpredictable risks need a general buffer of time and budget to survive them.

20 Explain risk identification, projection and refinement.

IMPORTANT

Risk identification is listing all possible risks for the project.

It is done by brainstorming with the team and using checklists and past experience.

Risk projection estimates each risk in two ways: how likely it is and how much damage it causes.

A risk with high probability and high impact becomes the top priority.

Risk refinement breaks a big vague risk into smaller detailed ones.

For example, staff problems is refined into no training, poor communication and low motivation.

Detailed risks are easier to plan for than one big vague worry.

These three steps turn vague worries into a managed risk list the team can act on.

21 What is a risk table? How is it used?

IMPORTANT

A risk table is a list of all identified risks with their probability and impact level.

Each row has the risk name, chance of happening and damage if it happens.

Risks are sorted so the team handles the biggest ones first.

Example: an 80 percent chance of a 2-week delay ranks above a 10 percent chance of a 1-day delay.

The table is reviewed in every project meeting to check if risks are growing or shrinking.

New risks found during the project are added to the table.

It keeps risk handling organised instead of depending on memory.

Managers use it to decide where to spend time and money on risk control.

22 What is RMMM? Explain its four parts.

IMPORTANT

RMMM stands for Risk Mitigation, Monitoring and Management.

It is the plan for handling each major risk in the project.

Mitigation: steps taken to avoid the risk or reduce its damage before it happens.

Monitoring: watching for warning signs that the risk is coming closer.

Management: the backup plan executed if the risk actually happens.

Together they cover the risk before, during and after it occurs.

Every major risk in the risk table gets its own RMMM plan.

RMMM turns risk from a fear into a managed, documented process.

23 Prepare an RMMM plan for the risk of losing a key developer.

IMPORTANT

Risk: the key developer working on the payment module may leave mid-project.

Mitigation: train a second developer on the critical modules right now so knowledge is shared.

Also document the module design and code clearly so a newcomer can understand it fast.

Monitoring: watch for warning signs like falling motivation, frequent leave or job interviews.

Hold regular one-to-one talks to catch dissatisfaction early.

Management: if the developer leaves, start the replacement hiring process within one week.

The trained second developer takes over temporarily so work does not stop.

This plan keeps the project safe even if the worst happens.

24 Distinguish between risk mitigation and risk management.

IMPORTANT

Mitigation is proactive: it acts before the risk happens to avoid or soften it.

Management is the backup action: it acts after the risk happens to recover.

Example: taking daily data backups is mitigation because it reduces damage.

Restoring the system from backup after a crash is management because it fixes the damage.

Mitigation reduces the chance or impact; management reduces the loss after impact.

Both are written in the RMMM plan for every major risk.

A project with only mitigation has no backup, and one with only management suffers full damage first.

So both are needed for complete risk control.

25 What are the contents of an RMMM plan?

IMPORTANT

An RMMM plan documents how each major risk will be handled.

It lists each major risk with its mitigation steps to avoid or reduce it.

It lists the monitoring indicators, the warning signs to watch for each risk.

It lists the management actions, the backup plan to run if the risk occurs.

It names who is responsible for watching and handling each risk.

It also notes the cost and time needed for each mitigation and backup action.

The plan is reviewed regularly because new risks appear during the project.

A complete RMMM plan means no major risk is left to chance.

26 Explain software pricing and its approaches.

IMPORTANT

Software pricing decides how much to charge the customer for building the software.

Cost-based pricing estimates the total cost and then adds a profit margin on top.

Value-based pricing charges based on the benefit the customer gets from the software.

Example: a billing system that saves the customer 10 lakhs a year can be priced higher than its build cost.

Market-based pricing also looks at what competitors charge for similar work.

Pricing must consider effort, risk, hardware and future maintenance costs.

The final price is quoted in the contract before the project starts.

Good pricing keeps the company profitable while winning the contract.

27 What are the risks of wrong software pricing?

IMPORTANT

Underpricing means the company loses money even if the project succeeds.

Overpricing means the customer rejects the quote and the contract goes to competitors.

Ignoring maintenance cost causes losses after delivery when bugs must be fixed free.

Ignoring risk means unexpected problems eat up all the profit.

For example, a fixed-price quote without a risk buffer can turn into a loss if requirements change.

Wrong pricing can also damage the company reputation if quality is cut to save cost.

So pricing mistakes hurt money, customers and reputation together.

Good pricing balances cost, value, risk and competition carefully.

28 **What is plan-driven development? [Asked: Model paper]**

REPEATED

Plan-driven development creates the complete plan, schedule and documents before coding starts.

All requirements are fixed first, then design, then implementation in order.

It suits projects with stable requirements and strict deadlines.

It gives the customer clear dates, costs and documents at every stage.

The waterfall model is the classic example of plan-driven development.

Its weakness is that changing requirements mid-way is costly and difficult.

It works best for critical systems like banking or medical software needing full documentation.

So plan-driven means plan everything first, then build exactly as planned.

29 **Describe project planning in software engineering. [Asked: Model paper]**

REPEATED

Project planning is the activity of deciding how a software project will be executed before the work starts.

It sets the project scope: what will be built and what will not.

It breaks the work into tasks and assigns them to team members with deadlines.

It estimates the effort, cost and time needed, using techniques like COCOMO or expert judgement.

It identifies risks and prepares responses for them in advance.

It creates a schedule, often shown as a Gantt chart, so progress can be tracked week by week.

It plans quality checks, reviews and testing at the right points in the timeline.

It also plans the team structure, tools and communication methods.

Example: before building a school fee portal, the plan lists modules, assigns two developers, estimates three months, and schedules testing in the final two weeks.

In short, project planning turns a vague idea into a scheduled, costed and risk-aware path to delivery.

30 What is project scheduling? Explain with Gantt chart.

IMPORTANT

Project scheduling divides the project into tasks with durations and deadlines.

Each task is assigned people, resources and a start and end date.

Tasks are ordered by dependencies: some tasks must finish before others begin.

A Gantt chart shows this schedule as horizontal bars on a timeline.

Each bar shows a task, its length shows duration, and its position shows timing.

The manager tracks progress by comparing actual work against the chart.

Delays are visible at once because bars shift to the right.

Good scheduling keeps everyone aware of what to do and when to finish it.

31 What is the critical path in project scheduling?

IMPORTANT

The critical path is the longest sequence of dependent tasks in the project.

It decides the shortest possible time in which the project can finish.

If any task on this path is delayed, the whole project is delayed.

Tasks not on the critical path have slack, meaning they can be delayed a little without harm.

Managers watch critical path tasks most closely and give them the best people.

PERT charts help find the critical path by showing task dependencies.

For example, in a college portal project, database design may lie on the critical path.

Protecting the critical path is the key to delivering on time.

32 What is the purpose of a feasibility study?

IMPORTANT

A feasibility study checks whether a proposed software project is worth doing before money is spent.

Technical feasibility asks whether the team has the technology and skill to build it.

Economic feasibility asks whether the benefits will be greater than the cost.

Operational feasibility asks whether the organisation can actually use and maintain the system.

Legal feasibility checks that the project breaks no laws, licences or contracts.

Schedule feasibility checks whether it can be finished in the available time.

If any dimension fails badly, the project should be reshaped or cancelled early.

Example: before building a drone-delivery app, the study might find that local aviation rules

make it legally infeasible.

In short, a feasibility study answers should we build this at all before the team commits.

33 Distinguish between deliverables and milestones.

IMPORTANT

Deliverables are the actual products handed over; milestones are the checkpoints that mark progress.

A deliverable is something the customer receives, like a tested module, a design document or a user manual.

A milestone is a point in time when a major result is achieved, like completing the design phase.

Deliverables are judged by quality; milestones are judged by date.

Every milestone usually produces one or more deliverables as proof.

Missing a deliverable means the work is incomplete; missing a milestone means the schedule has slipped.

Both are fixed in the project plan so progress is visible and measurable.

Example: a tested login module is a deliverable, while login module completed by 10 March is a milestone.

In short, deliverables are what you give, milestones are when you prove progress.

34 Explain agile planning.

IMPORTANT

Agile planning plans only as far as needed and then adapts as work continues.

Instead of one big plan, planning happens at three levels.

Release planning decides which features go into the next release.

Iteration planning picks the work for the next sprint, usually two weeks.

Daily planning happens in the standup meeting where the team adjusts the day's work.

Plans are updated every iteration based on feedback and velocity.

Velocity is the amount of work finished per sprint, and it guides how much to promise next time.

So agile planning stays flexible and always reflects the latest reality.

35 **Compare traditional and agile project planning.**

IMPORTANT

Traditional planning makes one big plan at the start and changing it is hard.

Agile planning makes small plans every iteration and changing them is easy.

Traditional plans are detailed and document-heavy; agile plans are light and visual.

Traditional suits fixed scope projects with stable requirements.

Agile suits evolving scope where customer feedback keeps changing priorities.

In traditional planning, delays are found late; in agile, velocity shows problems every sprint.

For example, a government tender project uses traditional planning while a startup app uses agile.

The choice depends on how stable the requirements are.

36 **Write a short note on the COCOMO model. [Asked: May 2019, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

COCOMO stands for Constructive Cost Model, created by Barry Boehm.

It estimates effort, time and cost from the estimated size of the software in lines of code.

It uses mathematical formulas tuned with data from many past projects.

There are three levels: basic for quick estimates, intermediate with cost factors, and detailed for each phase.

It also classifies projects as organic, semi-detached or embedded based on complexity.

COCOMO suits large plan-driven projects better than small agile ones.

Its accuracy depends on how correctly the size is estimated at the start.

It remains one of the most famous algorithmic estimation models in software engineering.

37 What is estimation? Explain the estimation techniques. [Asked: Model paper]

REPEATED

Estimation predicts how much effort, time and cost a project needs before building starts.

Expert judgement asks experienced people to guess based on similar past projects.

Analogy compares the new project with a completed one and adjusts the numbers.

The COCOMO model uses formulas based on the estimated size of the software.

Story points let agile teams estimate task size relatively instead of in hours.

Good estimation needs historical data from similar projects.

Bad estimation causes missed deadlines, budget overruns and unhappy customers.

So estimation is a skill every project manager must master.

38 Explain function point analysis. [Asked: Oct 2021, Aug 2022]

REPEATED

Function point analysis is an estimation technique that measures software size by its functionality, not by lines of code.

It counts five functional units: external inputs, external outputs, external inquiries, internal logical files and external interface files.

Each unit is rated as simple, average or complex and given a weight.

The weighted sum gives the unadjusted function point count.

This count is then adjusted using fourteen general system characteristics, such as performance and reusability needs.

The final function point total is converted into effort and cost using the organisation's past project data.

Because it needs no code, it can estimate very early, right after requirements are known.

Example: a student record system with 10 inputs, 8 outputs and 3 files might total 120 function points, which past projects show needs about 4 person-months.

In short, function points estimate size from what the software does, making early estimates possible.

39 What are story points in agile estimation?

IMPORTANT

Story points measure task size relatively instead of in hours.

The team compares a new task with completed ones: this task is twice as big as that one.

A simple 1, 2, 3, 5, 8 scale is commonly used, like the Fibonacci series.

Points cover effort, complexity and uncertainty together, not just time.

After a few sprints, the team knows its velocity: the points finished per sprint.

Velocity is used to plan how many points the team can take in future sprints.

For example, a team with velocity 20 will not promise 30 points next sprint.

Story points make agile planning realistic and free from false hour promises.

MODULE 3

Design, Testing and Metrics

3

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q8-Q10 come from this module (design and testing). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2; the model paper essay is system testing approaches - the single most likely 15-marker. White-box techniques 5x, black-box techniques 3x, cohesion-coupling 3x, reliability 3x - all sure marks. UML diagrams have zero old-paper precedent but are core 2024-syllabus topics; attempt at least one question from each course outcome (CO3 = this module).

1 What is software design? Describe its objectives. [Asked: Aug 2022, Feb 2025]

REPEATED

Software design is the process of planning how the software will be built before coding starts.

Its first objective is to create a blueprint that developers can follow without confusion.

It aims for correctness: the design must faithfully reflect the requirements.

It aims for modularity: breaking the system into independent, manageable parts.

It aims to make future changes cheap by keeping parts loosely connected.

It aims for understandability, so new team members can grasp the system quickly.

It produces documents like architecture diagrams and interface designs that guide coding and testing.

Example: designing a bus-tracking app means deciding its modules, screens, database tables and how GPS data flows, all before writing code.

In short, software design turns requirements into a clear construction plan.

2 Explain the software design process.

IMPORTANT

Software design is the step where the requirements are converted into a clear plan for building the software.

It answers the question: how will the software be built, before any coding starts.

Step 1: Study the requirements document carefully so that nothing is missed.

Step 2: Create the high-level architecture, dividing the system into major parts.

Step 3: Design the data, deciding the tables, files and data structures to use.

Step 4: Design each component in detail, defining its inputs, outputs and internal logic.

Step 5: Design the user interface, deciding the screens and how the user interacts with them.

Step 6: Review the design against the requirements and correct any mistakes.

Only after the design is reviewed and approved does coding begin.

Example: before coding a college portal, designers first draw how the login, fees and exam modules connect.

A good design process saves time, because fixing a design mistake is much cheaper than fixing code later.

3 **What are the inputs and outputs of the design process?**

IMPORTANT

The input of the design process is the approved software requirements document.

It also takes constraints as input, such as the hardware limits, budget, deadline and technology to be used.

The output of the design process is a complete set of design documents.

These include architecture diagrams showing the major parts and their connections.

They also include data design documents, such as database tables and their relationships.

Interface design documents describe the screens and the way modules communicate.

Component design documents describe the internal logic of each module in detail.

The design documents act as a blueprint that programmers follow while coding.

Before coding starts, the design output is reviewed by the team for errors and gaps.

Example: for a banking app, the output would include the database design for accounts and the screen design for money transfer.

In short, requirements go in as input, and a buildable design comes out as output.

4 **Define reliability. Explain the ways of characterising failure occurrences in time. [Asked: Mar 2021, Oct 2021, Aug 2022]**

SURE MARK

Reliability is the probability that software works without failure for a given period in a given environment.

It is measured by observing how failures occur over time, not by guessing.

There are four general ways of characterising failure occurrences in time.

Time of failure records exactly when each failure happens.

Time interval between failures measures the gaps between failures, which grow as the software improves.

Cumulative failures plot the total count over time, showing whether failures are slowing down.

Failures experienced in a time interval show the failure rate in each period, like failures per week.

These measures tell the team whether testing is working and when the software is ready to release.

Example: if a server fails after 10, 40 and 100 days, the growing intervals show reliability is improving as bugs are fixed.

In short, reliability is quantified by studying the timing pattern of failures.

5 What are the characteristics of a good design?

IMPORTANT

A good design is one that is correct, easy to understand, easy to change and easy to test.

Correctness: the design fully covers all the requirements with nothing missing or extra.

Modularity: the system is split into independent modules, so one part can be changed without breaking others.

Low coupling: modules depend on each other as little as possible.

High cohesion: each module does one clear job and does it completely.

Understandability: another developer can read the design and understand it without much help.

Simplicity: the design avoids unnecessary complexity; the simplest solution that works is best.

Testability: the design makes it easy to test each module separately.

Reusability: well-designed modules can be reused in future projects.

Example: a good design for an online shop keeps payment, orders and stock as separate modules.

Good design is the foundation; bad design leads to bugs, delays and painful maintenance.

6 How is design quality reviewed before coding?

IMPORTANT

Design quality is checked before coding through a design review meeting.

In a review, the designers present their design documents to a group of reviewers.

Reviewers check whether every requirement is covered by the design.

They look for missing parts, unclear modules and risky decisions.

They check coupling and cohesion, making sure modules are independent and focused.

They also check that the design follows the project standards and constraints.

Any defect found is noted down and the design is corrected before coding starts.

Finding a defect in design review is far cheaper than finding it in code or after delivery.

Example: reviewers may notice that the login module was never connected to the user database, and fix it early.

In short, design review is a quality gate that stops bad design from reaching the coding stage.

7 **Explain abstraction, modularity, coupling and cohesion. [Asked: Mar 2021, Oct 2021, Aug 2022]**

SURE MARK

These four concepts are the basic rules that guide every good designer.

Abstraction means hiding complex details and showing only what is needed.

It lets a designer work with a simple view of a module without worrying about its inner code.

Modularity means splitting the system into separate independent modules.

Each module can be developed, tested and changed on its own.

Coupling is the degree of dependence between modules; good design keeps coupling low.

With low coupling, changing one module does not break the others.

Cohesion is how focused a single module is; good design keeps cohesion high.

With high cohesion, each module does one clear job completely.

Example: a login module should only handle login, and should not depend on the report module.

In short, good design means simple views, separate parts, low dependence and focused modules.

8 **What is cohesion? Why should it be high? [Asked: Model paper]**

REPEATED

Cohesion is the measure of how focused a single module is on one job.

In high cohesion, all the parts of a module work together for one clear purpose.

A highly cohesive module is easy to understand, because it does just one thing.

It is also easy to test and reuse, since it has a single clear responsibility.

In low cohesion, a module does many unrelated jobs mixed together.

Low cohesion modules are confusing, hard to test and hard to change safely.

Cohesion should be high because a focused module is reliable and easy to maintain.

Example: a payment module that only handles payments has high cohesion.

If the same module also prints reports and manages user accounts, it has low cohesion.

That is why designers always aim for high cohesion inside each module.

9 What is coupling? Why should it be low?

IMPORTANT

Coupling is the measure of how much different modules depend on each other.

In high coupling, modules are tightly connected and depend on each other's internal details.

With high coupling, changing one module can break another module, even one that seems unrelated.

High coupling also makes testing hard, because a module cannot be tested alone.

In low coupling, modules interact only through simple, well-defined interfaces.

With low coupling, each module can be changed, tested and reused independently.

Coupling should be low because software always changes, and low coupling keeps changes safe and cheap.

Example: if the billing module directly reads the internals of the stock module, they are highly coupled.

If instead billing only calls a simple getPrice() function, the coupling is low.

That is why designers always aim for low coupling between modules.

10 Write about the term module.

IMPORTANT

A module is a separate, independent unit of software that performs one clear task.

Good modules have high cohesion: everything inside works toward one single purpose.

Good modules have low coupling: they depend as little as possible on other modules.

Modules communicate through well-defined interfaces while hiding their internal details.

Breaking software into modules makes it easier to understand, test and change.

A faulty module can be fixed or replaced without rewriting the whole system.

Modules also enable teamwork, since different people can build different modules in parallel.

Example: in a college app, admission, fees and attendance can be three separate modules.

In short, a module is one well-shaped brick, and good software is built from such bricks.

11 Explain the difference between object-oriented and function-oriented design.

IMPORTANT

Object-oriented design organises software around objects; function-oriented design organises it around functions.

In object-oriented design, data and the operations on that data are bundled together into classes.

In function-oriented design, the system is broken into functions that transform input data into output data.

Object-oriented design uses inheritance and polymorphism to reuse code.

Function-oriented design uses data flow diagrams and structure charts to show processing steps.

Object-oriented design suits systems with complex, long-lived data, like banking systems.

Function-oriented design suits systems that are mainly data processing pipelines, like payroll.

Maintenance differs too: objects localise change, while functions may need changes across many steps.

Example: a game with players and enemies fits objects; a salary-slip generator fits functions.

In short, objects group data with behaviour, functions group the steps that process data.

12 Explain the design model and its levels.

IMPORTANT

The design model is the complete set of design documents that describe the software.

It is created in levels, each level adding more detail than the previous one.

Data design: decides the data structures, database tables, fields and relationships.

Architectural design: decides the overall structure, the major parts and how they connect.

Interface design: decides the user screens and the way different parts communicate with each other.

Component design: decides the internal logic of each part, close to the actual code.

The levels move from the big picture down to the fine details.

Each level is reviewed before moving to the next, so mistakes are caught early.

Example: for a hospital system, architecture first splits it into patient, doctor and billing parts, then each part is designed in detail.

In short, the design model is a layered blueprint that guides the programmers step by step.

13 Why is the design model created in levels?

IMPORTANT

The design model is created in levels because each level hides detail and shows one clear view.

Reviewers can focus on one view at a time instead of being confused by everything at once.

The high-level architecture is agreed first, before time is spent on small details.

This saves effort, because a mistake at a high level would waste all the detailed work below it.

Changes at a high level are cheap, while changes in finished code are costly.

Levels also let different people work in parallel: one team on architecture, another on data design.

Each level becomes a document that guides the next level of work.

Example: if the architecture of a bank app is wrong, every module built on it would need rework.

So the team first agrees on the architecture, and only then designs each component in detail.

In short, levels make design easier to review, cheaper to change and safer to build on.

14 What is software architecture? Why is it important?

IMPORTANT

Software architecture is the high-level structure of the system.

It defines the major components of the software and how they connect and communicate.

It is decided early in the project, before detailed design and coding.

Architecture is important because it is very hard and costly to change later.

Every module is built on top of the architecture, so a wrong architecture affects everything.

A good architecture makes the system scalable, so it can grow with more users.

It also makes the system maintainable, so future changes are easy.

It helps the team divide work, since each component can be built by a different group.

Example: a college portal may use client-server architecture, with a web front end and a database back end.

In short, architecture is the skeleton of the software; a strong skeleton supports a strong system.

15 Explain any three architectural styles with examples.

IMPORTANT

An architectural style is a standard pattern for organising the parts of a system.

Layered style: the system is split into layers like presentation, business logic and data.

Each layer talks only to the layer below it, which keeps the design clean.

Example: a typical web app has a browser layer, a server logic layer and a database layer.

Client-server style: clients send requests and the server processes them and replies.

The server holds the data and logic, while clients only show the interface.

Example: an email system where the phone app is the client and Gmail servers do the work.

MVC style: the system is split into Model (data), View (screen) and Controller (logic).

The Model stores data, the View shows it, and the Controller handles user actions.

Example: many web frameworks build apps with a separate model, view and controller.

Each style suits different problems, and the architect picks the one that fits best.

16 What is data design?

IMPORTANT

Data design is the part of design that decides how data is organised and stored.

It is done early, before the detailed design of components.

It defines the data structures the program will use, such as lists, trees and objects.

For database systems, it defines the tables, their fields and the relationships between them.

It also decides which data is stored permanently and which is temporary.

Good data design makes the software fast, because data is easy to find.

It also makes the software reliable, because data stays consistent and safe.

Programmers use the data design as a guide when they write code that handles data.

Example: a library system needs tables for books, members and loans, linked by book id and member id.

In short, data design is the plan for what data exists and where it lives.

17 Give two qualities of a good data design.

IMPORTANT

A good data design has two main qualities: no duplication and fast access.

No duplication means the same fact is stored in only one place.

This avoids inconsistency, where two copies of the same data disagree with each other.

It also saves storage space and makes updates easy, since only one copy needs changing.

Example: a customer address is stored once in the customer table, not copied into every order.

Fast access means frequently used data is organised so it can be found quickly.

This is done using indexes, proper keys and well-planned table structures.

Fast access keeps the software responsive even when the data grows large.

Example: indexing the roll number field lets the system find a student record instantly.

In short, good data design stores each fact once and finds it fast.

18 What is architectural design? How is it different from detailed design?

IMPORTANT

Architectural design defines the big parts of the system and how they connect.

It answers: what are the main components, and how do they talk to each other?

It focuses on structure, not on the internal code logic of any part.

Its output is usually shown with block diagrams of components and their links.

Detailed design, on the other hand, describes the inside of each component.

It defines the algorithms, data structures and step-by-step logic of every module.

Architectural design comes first; detailed design comes after it.

In short, architecture decides the shape of the building, while detailed design decides the wiring of each room.

Example: an online shop is first split into user interface, order processing and payment components.

Then detailed design describes exactly how the payment component verifies a card.

Both are needed: architecture gives direction, detailed design gives the code-level plan.

19 Why is architectural design hard to change later?

IMPORTANT

Architectural design is hard to change later because everything is built on top of it.

Every component depends on the architecture for how it connects to other components.

Changing the architecture means changing the interfaces between many components.

That forces rewriting, retesting and re-integrating large parts of the system.

The later the change, the more work is already done, and the more expensive the rework.

Changing architecture late can also break things that were working fine.

This is why architecture is reviewed very carefully before construction starts.

Example: switching a system from one central server to microservices after coding means rebuilding almost everything.

In short, the foundation of a building cannot be changed after the floors are built.

So architects take extra care to get the architecture right the first time.

20 Explain the class diagram with its notation.

IMPORTANT

A class diagram shows the classes in the system and the relationships between them.

It is the most used UML diagram for object-oriented design.

Each class is drawn as a box divided into three parts.

The top part holds the class name, written in the centre.

The middle part lists the attributes, which are the data the class stores.

The bottom part lists the methods, which are the actions the class can do.

Attributes and methods can show visibility, like + for public and - for private.

Lines between boxes show relationships such as association, inheritance and aggregation.

Example: a Student class box has name and rollNo as attributes and register() as a method.

The diagram gives programmers a clear map of all classes before coding starts.

In short, a class diagram is the blueprint of the object-oriented structure of the software.

21 Explain the relationships in a class diagram with symbols.

IMPORTANT

Association means one class uses another class; it is shown by a plain straight line.

The line may have numbers at the ends showing how many objects are linked.

Example: a Student is associated with a Course, since a student enrolls in courses.

Inheritance means a child class extends a parent class, an is-a relationship.

It is shown by a solid line with a hollow arrow pointing to the parent class.

Example: Car inherits from Vehicle, since a car is a vehicle.

Aggregation means a whole-part relationship, a has-a relationship.

It is shown by a solid line with a hollow diamond at the whole end.

Example: a Library has Books; the books can exist even without that library.

Dependency means one class briefly needs another, shown by a dashed arrow.

Example: a Report class depends on a Printer class only while printing.

Knowing these symbols lets a designer read any class diagram correctly.

22 Draw and explain a class diagram for a library Book class.

IMPORTANT

To draw the Book class, draw a rectangle box divided into three horizontal parts.

Top part: write the class name, Book, in the centre.

Middle part: list the attributes, which are the data of a book.

Attributes: title, author, price and isIssued, each with its data type.

Bottom part: list the methods, which are the actions a book object can do.

Methods: issue(), returnBook() and displayDetails().

Visibility can be shown: - for private attributes and + for public methods.

If the library has many books, draw an association line from Library to Book with 1 to many multiplicity.

The diagram tells the programmer exactly what data and actions the Book class must have.

Example code-level reading: book.issue() sets isIssued to true when a member borrows it.

In short, this one box is the complete design of the Book class.

23 Explain the sequence diagram with its notation.

IMPORTANT

A sequence diagram shows how objects interact with each other over time.

It is perfect for showing one scenario, like a login flow, step by step.

Time runs from top to bottom in the diagram.

Objects are drawn as boxes at the top of the diagram.

From each object, a dashed vertical line called a lifeline runs downward.

Messages between objects are drawn as horizontal arrows from one lifeline to another.

The arrows are labelled with the message name and are placed in time order.

A reply message is usually shown with a dashed arrow going back.

A tall thin rectangle on a lifeline shows when an object is active.

Example: the User sends login() to the LoginPage, which sends verify() to the Database, which returns the result.

In short, a sequence diagram is a timeline of who talks to whom and in what order.

24 Draw a sequence diagram for a login process.

IMPORTANT

To draw the login sequence, place three boxes at the top: User, LoginPage and Database.

Draw a dashed lifeline going down from each box.

Arrow 1: from User to LoginPage, labelled login(username, password).

Arrow 2: from LoginPage to Database, labelled verify(username, password).

Arrow 3: a dashed reply arrow from Database back to LoginPage, labelled result (valid or invalid).

Arrow 4: from LoginPage back to User, labelled login success or login failed.

If the login fails, an alternative block can show a retry message.

The arrows are read top to bottom, which is the exact order of events.

This diagram shows the programmer the exact steps to code for the login process.

In short, it is a clear visual script of what happens during login.

25 Distinguish between sequence and collaboration diagrams.

IMPORTANT

Both diagrams show the same object interactions, but they organise the information differently.

A sequence diagram shows the order of messages over time.

Time flows top to bottom, so the sequence of events is very clear.

It is best when the order of steps matters, like in a login flow.

A collaboration diagram shows how the objects are linked to each other.

Objects are placed freely and connected by lines showing their links.

Messages are written on the links with numbers (1, 2, 3) to show the order.

It is best when the structure of connections matters more than the timeline.

In short, sequence focuses on time order, while collaboration focuses on object links.

A designer can convert one into the other since they carry the same information.

26 Explain the use case diagram with its notation.

IMPORTANT

A use case diagram shows what the system offers to its users.

It captures the functional requirements from the user's point of view.

Actors are drawn as stick figures outside the system boundary.

An actor is anyone or anything that interacts with the system, like a customer or another system.

Use cases are drawn as ovals inside the system boundary box.

Each oval is one goal the actor wants to achieve, like Place Order.

Lines connect each actor to the use cases they take part in.

The include relationship shows a use case that must always be used, drawn with a dashed arrow.

The extend relationship shows an optional use case, also drawn with a dashed arrow.

Example: in a bank system, Customer is an actor and Withdraw Cash is a use case.

In short, the diagram is a simple picture of who can do what with the system.

27 Draw a use case diagram for an online shopping system.

IMPORTANT

Draw a system boundary box and label it Online Shop.

Place two actors outside the box: Customer on the left and Admin on the right.

Inside the box, draw four ovals: Browse Products, Place Order, Make Payment and Manage Stock.

Connect the Customer actor with lines to Browse Products, Place Order and Make Payment.

Connect the Admin actor with a line to Manage Stock.

Add a use case Verify Card inside the box, connected from Make Payment with an include arrow.

This shows that every payment must include card verification.

The diagram clearly shows that customers shop and pay, while the admin manages stock.

It helps the team agree on the system's features before detailed design.

In short, this one picture captures the main functions of the online shop.

28 What is an actor in a use case diagram? Give examples.

IMPORTANT

An actor is a role played by a user or another system that interacts with the system.

It is drawn as a stick figure outside the system boundary.

An actor is not a specific person; it is a role.

The same person can play different actors at different times.

Actors can be primary, like a customer who uses the system directly.

Actors can also be secondary, like a bank server that the system talks to.

Each actor is connected to the use cases it takes part in.

Example: in a college system, Student and Teacher are actors.

A teacher can also be an Examiner actor when setting question papers.

In short, actors define who is outside the system and what they expect from it.

29 What are use case scenarios?

IMPORTANT

A use case scenario is a specific story of how a user interacts with the system to achieve a goal.

The use case names the goal, like withdraw cash; scenarios describe particular paths through it.

The main success scenario describes the happy path where everything works smoothly.

Alternate scenarios describe variations, like choosing a different account.

Exception scenarios describe what happens when things go wrong, like insufficient balance.

Scenarios are written in simple steps: the actor does this, the system responds with that.

They help designers cover every path and help testers derive test cases.

Example: for book a bus ticket, the main scenario is selecting a seat and paying; an exception scenario is payment failure with a retry option.

In short, scenarios turn an abstract use case into concrete stories the team can design and test against.

30 Explain the collaboration diagram.

IMPORTANT

A collaboration diagram shows how objects interact, focusing on their links.

Objects are drawn as boxes placed freely on the diagram.

Lines called links connect the objects that communicate with each other.

Messages are written on the links with numbers like 1, 2 and 3 to show the order.

It shows the same information as a sequence diagram but arranged by structure, not by time.

It is useful when the designer wants to see which objects are connected.

Example: for login, User, LoginPage and Database boxes are linked, with 1: login(), 2: verify(), 3: result.

The numbered messages let the reader follow the order of events.

In short, a collaboration diagram is a map of object links with ordered messages on them.

Designers often draw it when the connection structure matters most.

31 Explain the component diagram with an example.

IMPORTANT

A component diagram shows the physical parts of the system and their dependencies.

Physical parts include executables, libraries, files and database drivers.

Each component is drawn as a box, sometimes with a small component symbol.

Dependency arrows between boxes show which part needs which other part.

It helps plan how the system will be built, assembled and deployed.

It shows the impact of change: if one component changes, its dependents are affected.

Example: a billing app component depends on the payment library and the database driver.

If the payment library is updated, the diagram shows the billing app must be retested.

It is drawn late in design, when the physical structure of the code is known.

In short, a component diagram is a map of the buildable pieces and how they fit together.

32 Explain unit, integration, validation and system testing. [Asked: May 2019, Oct 2021]

REPEATED

Unit testing tests each small part of the software in isolation.

It is usually done by the developer who wrote the code.

It catches bugs early, when they are cheapest to fix.

Example: testing a single calculateTotal() function with different inputs.

Integration testing tests combined modules working together.

It checks that data passes correctly between modules.

Example: testing that the order module correctly calls the payment module.

Validation testing checks that the software meets the customer's requirements.

It answers: did we build the right product?

System testing tests the complete integrated system as one whole.

It is done in an environment similar to the real user setup.

Example: testing the full online shop from browsing to delivery on real servers.

In short, the levels move from small pieces to the whole system, catching different bugs at each step.

33 Explain verification and validation in detail. [Asked: Mar 2021, Aug 2022]

REPEATED

Verification and validation are the two complementary checks that software is built correctly.

Verification asks: are we building the product right? It checks work products against their specifications.

It includes reviews, inspections and walkthroughs, all done without running the code.

Validation asks: are we building the right product? It checks the software against user needs.

It is done by actually executing the software through testing.

Verification catches errors early in documents and design, when fixing them is cheap.

Validation catches errors in behaviour, when the software meets real users and real data.

Both are needed because a product can meet its specification yet still disappoint the user.

Example: verifying a design document for missing screens is verification; running the app to see if booking works is validation.

In short, verification checks the papers, validation checks the product, and both are needed for quality.

34 What are the objectives of software testing? [Asked: Model paper]

REPEATED

The main objective of testing is to find errors in the software before delivery.

It verifies that the software meets the requirements agreed with the customer.

It validates that the software behaves correctly under real conditions.

It builds confidence that the product is fit for use.

It measures quality by counting and classifying the defects found.

It prevents future failures by catching problems when they are still cheap to fix.

It also checks non-functional needs like speed, security and usability.

Example: the objective of testing a payment app is not to prove it works, but to hunt down every way money could go missing.

In short, testing aims to expose weaknesses early so the delivered software can be trusted.

35 What is a testing strategy? Explain the levels of testing.

IMPORTANT

A testing strategy is the overall plan that organises all testing work.

It answers: what to test, when to test, who tests it and what counts as pass.

Testing is done in levels, from small parts up to the full system.

Unit testing: each small module is tested alone by the developer.

Integration testing: combined modules are tested to check they work together.

Validation testing: the software is checked against the requirements.

System testing: the full system is tested end to end in a real-like environment.

A good strategy finds bugs early, when they are cheapest to fix.

It also defines the test environment, tools and resources needed.

Example: a bank app plan would test each function first, then the full transfer flow.

In short, the strategy is the roadmap that guides testing from code to delivery.

36 Is complete testing of a software possible? Explain why.

IMPORTANT

Complete testing of software is not possible in practice.

The number of possible inputs is usually infinite: every number, text and click combination cannot be tried.

All execution paths cannot be covered because loops and combinations multiply beyond any test budget.

Time and money are limited, so testing must stop at a reasonable point.

Instead of completeness, testers aim for risk-based coverage of the most important paths and inputs.

Techniques like equivalence partitioning and boundary value analysis choose a smart sample instead of everything.

The goal shifts from proving no bugs exist to finding the important bugs early.

Example: a simple calculator with two number fields has billions of input pairs; no team can test them all.

In short, exhaustive testing is a theoretical idea; real testing samples the most valuable cases within the available time.

37 What is software testing?

IMPORTANT

Software testing is the process of running a program to find errors before users do.

Its goal is to uncover defects in requirements, design and code.

Testing checks both that the software does what it should and that it does not do what it should not.

It is done at several levels: unit, integration, validation and system testing.

Both manual testing by people and automated testing by scripts are used.

Good testing needs planned test cases with expected results, not random clicking.

Testing also measures quality by recording and classifying the defects found.

Example: testing a login page means trying correct passwords, wrong passwords, empty fields and locked accounts.

In short, testing is the safety net that catches mistakes before the customer pays for them.

38 Which types of errors are detected in software testing?

IMPORTANT

Testing detects several types of errors at different levels of the software.

Requirements errors: the software was built from misunderstood or missing requirements.

Design errors: the structure is wrong, like a missing module or a bad interface.

Coding errors: bugs in program logic, wrong calculations or mishandled conditions.

Interface errors: modules fail to communicate correctly with each other.

Performance errors: the software is too slow or uses too much memory.

Documentation errors: the manual or help text does not match what the software does.

Security errors: the software allows unauthorised access or leaks data.

Example: a fee portal might calculate totals wrong, which is a coding error, or show a confusing receipt, which is an interface error.

In short, testing hunts errors all the way from misunderstood requirements down to wrong lines of code.

39 Explain test strategies for conventional software.

IMPORTANT

Test strategies for conventional software follow the levels in order: unit, integration, then system.

Unit testing is done first on each module separately.

Integration testing then combines the modules step by step.

Top-down integration starts from the main module and works downward.

Missing lower modules are replaced with stubs, which are dummy modules.

Bottom-up integration starts from the lowest modules and works upward.

Missing upper modules are replaced with drivers, which are dummy calling programs.

Big-bang integration combines everything at once and tests it together.

Big-bang is risky because when a failure happens, it is hard to find which module caused it.

Example: testing a main menu with stubs for payment and stock is top-down testing.

In short, gradual integration finds bugs early, while big-bang finds them late and painfully.

40 What are stubs and drivers in integration testing?

IMPORTANT

A stub is a dummy lower-level module used in top-down integration testing.

It pretends to be the real module and returns simple fixed answers.

This lets the upper modules be tested before the lower ones are coded.

A driver is a dummy calling program used in bottom-up integration testing.

It calls the lower module and passes test data to it.

This lets the lower modules be tested before the upper ones are ready.

Example: testing a main menu with a stub that pretends the payment module always says payment ok.

Example: testing a payment function with a driver program that calls it with test card numbers.

Stubs and drivers are thrown away once the real modules are ready.

In short, stubs stand in for modules below, and drivers stand in for modules above.

41 **Explain black box testing techniques with examples. [Asked: May 2019, Oct 2021, Aug 2022]**

SURE MARK

Black box testing uses techniques based on inputs and outputs, not code.

Equivalence partitioning divides inputs into groups that should behave the same.

The tester picks one value from each group, which reduces the number of tests.

Example: for an age field 18 to 60, groups are below 18, 18 to 60, and above 60.

Testing 15, 30 and 70 covers all three groups.

Boundary value analysis tests the values at the edges of each group.

Most bugs hide at boundaries, so these values are tested carefully.

Example: for a 1 to 100 field, test 0, 1, 2, 99, 100 and 101.

Another example: for age 18 to 60, test 17, 18, 60 and 61.

These techniques find input-handling bugs without ever reading the code.

In short, test one from each group and always test the edges.

42 **Explain white box testing techniques with examples. [Asked: May 2019, Mar 2021, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

White box testing uses techniques based on the internal code structure.

Statement coverage requires every line of code to run at least once.

It is the weakest measure; running a line does not mean it is correct.

Branch coverage requires every decision outcome to be tested, both true and false.

Example: an if-else needs two tests, one for each branch.

Path coverage requires every possible route through the code to be tested.

It is the strongest but also the hardest, since paths can be very many.

Condition coverage tests each individual condition inside a decision.

Example: for if (a > 0 and b > 0), test all combinations of a and b.

These techniques need the tester to read and understand the code.

In short, white box testing makes sure no part of the code is left untested.

43 Distinguish between black box and white box testing.

IMPORTANT

Black box testing checks the software's behaviour without looking at the code.

The tester knows only the inputs and the expected outputs.

It is done from the user's point of view.

It finds missing functions, wrong outputs and interface errors.

Example: testing login by trying correct and wrong passwords.

White box testing checks the internal logic and structure of the code.

The tester sees the code and designs tests for its paths and branches.

It is done from the developer's point of view and needs code knowledge.

It finds hidden logic errors, untested paths and dead code.

Example: testing every if-else branch inside the login function.

In short, black box asks what the software does, while white box asks how it does it.

Both are needed because neither one alone finds all types of bugs.

44 Why are both black box and white box testing needed?

IMPORTANT

Black box and white box testing find different types of bugs, so both are needed.

Black box testing finds missing functions and wrong behaviour.

It checks the software from the user's view, so it catches usability and requirement errors.

Example: black box testing reveals that the forgot password feature is missing.

White box testing finds hidden logic errors inside the code.

It checks paths and branches that black box tests may never reach.

Example: white box testing reveals a division by zero in a rarely used branch.

Black box alone cannot find code-level bugs; white box alone cannot find missing features.

Using both gives the best defect detection and higher confidence.

In short, black box checks what the software does, white box checks how, and together they cover everything.

45 What is meant by loop testing? Explain.

IMPORTANT

Loop testing is a white-box technique that focuses on the validity of loop constructs in code.

Simple loops are tested with five cases: skip the loop entirely, one pass, two passes, a typical number of passes, and the maximum passes.

Nested loops are tested from the innermost loop outward, keeping outer loops at minimum values.

Concatenated loops are tested as separate simple loops when they are independent.

Unstructured loops are redesigned into structured form before testing wherever possible.

The aim is to catch off-by-one errors and wrong termination conditions.

Example: a loop that processes 100 student records is tested with 0, 1, 2, 50 and 100 records.

In short, loop testing systematically exercises loop boundaries, where many defects hide.

46 Explain flow graph with an example.

IMPORTANT

A flow graph is a diagram that shows the logical flow of a program for white-box testing.

Each circle, called a node, represents one or more program statements.

Arrows, called edges, show the flow of control from one node to another.

Areas bounded by edges and nodes are called regions, including the outside area.

The cyclomatic complexity is computed as edges minus nodes plus 2, giving the number of independent paths.

Each independent path becomes a test case in path testing.

Example: for a simple if-else that checks pass or fail marks, the flow graph has 4 nodes and the complexity is 2, so two test cases cover both paths.

In short, a flow graph turns code structure into a map that tells testers exactly which paths to test.

47 What is system testing? What does it include? [Asked: Model paper]

SURE MARK

System testing checks the fully integrated system as one complete whole.

It is done after integration testing, on the entire software plus hardware.

It is performed in an environment similar to the real user setup.

It includes performance testing, which checks speed under normal and heavy load.

It includes security testing, which checks that data is protected from attacks.

It includes recovery testing, which checks the system restarts correctly after a crash.

It also includes stress testing, usability testing and compatibility testing.

The goal is to prove the whole system works in real-world conditions.

Example: testing a railway booking site with thousands of users booking at once.

In short, system testing is the final check that the complete system is ready for the customer.

48 Explain validation testing with alpha and beta testing. [Asked: May 2019, Aug 2022]

REPEATED

Validation testing checks that the software meets the customer's requirements.

It answers: did we build the right product?

It is done after integration testing, on the nearly finished software.

Alpha testing is done by developers at the developer's site with users present.

Users try the software while the team watches and notes problems.

It happens before the software is released to the public.

Beta testing is done by real users at their own place.

The software is given to selected users who report bugs from daily use.

Example: a new app given to 100 students before launch is beta testing.

Alpha finds bugs in a controlled setting; beta finds bugs in real-world use.

In short, validation with alpha and beta testing proves the software is ready for release.

49 Explain functional testing and acceptance testing. [Asked: Mar 2021, Feb 2025]

REPEATED

Functional testing checks what the software does; acceptance testing checks whether the customer accepts it.

Functional testing verifies each function against its specification, using black-box techniques like equivalence partitioning.

It is usually done by the test team during system testing.

Acceptance testing is done by the customer or end users before final delivery.

Alpha testing is acceptance testing done at the developer's site with the customer present.

Beta testing is acceptance testing done at customer sites under real conditions.

Functional testing can pass while acceptance testing fails, if the software is correct but unusable.

Example: functional testing checks that a 10 percent discount applies correctly; acceptance testing is the shop owner confirming the billing screen is usable for daily work.

In short, functional testing proves the features work, acceptance testing proves the customer is satisfied.

50 What is recovery testing? Explain.

IMPORTANT

Recovery testing checks whether software can recover correctly after a failure.

The system is forced to fail in various ways: power cut, network drop or disk full.

Testers then check that it restarts and continues without data loss or corruption.

It verifies that recovery procedures like backups, checkpoints and transaction rollbacks actually work.

It also checks that the system returns to a known good state within an acceptable time.

It is a type of system testing focused on reliability rather than features.

Example: a banking app is tested by cutting the network during a transfer; it must roll back cleanly so no money is lost or duplicated.

In short, recovery testing proves the software can fall down safely and get back up.

51 What is robustness testing? Explain.

IMPORTANT

Robustness testing checks how software behaves with invalid or unexpected inputs.

It feeds wrong data types, extreme values and empty inputs to see whether the program crashes.

The software should handle bad input gracefully with clear error messages, not freeze or corrupt data.

It is a black-box technique because it needs no knowledge of the code inside.

It protects against both mistaken users and deliberate misuse.

Test cases include boundary breakers like huge numbers, special characters and null values.

Example: entering letters in a phone-number field or a birth year of 3000 should show a polite error, not crash the app.

In short, robustness testing proves the software stays standing when users do unexpected things.

52 What is debugging? Explain the debugging process.

IMPORTANT

Debugging is the process of finding the cause of an error and fixing it.

It starts after testing reports a failure.

Step 1: reproduce the bug, so it can be seen happening again.

Step 2: locate the cause by studying the code around the failure.

Step 3: fix the faulty code with the smallest safe change.

Step 4: retest to confirm the bug is gone and nothing else broke.

Common techniques include adding print statements to trace values.

Breakpoints let the developer pause the program and inspect variables.

Backtracking means going backwards from the failure to find its origin.

Example: a wrong total in a bill is traced back to a faulty discount calculation.

In short, debugging is detective work: find the cause, fix it, and prove it is fixed.

53 Distinguish between testing and debugging.

IMPORTANT

Testing and debugging are related but different activities.

Testing is the process of running the software to find bugs.

It shows that a bug exists by running test cases with known inputs.

Testing can be done by testers as well as developers.

Debugging is the process of finding where the bug is and fixing it.

It starts after testing reports a failure.

Debugging is done by developers, since it needs code knowledge.

In short, testing finds the symptom, while debugging finds and cures the disease.

Example: testing shows the login fails; debugging finds the wrong password check in the code.

A good team does both well: testing to catch bugs, debugging to remove them.

54 What is the significance of design model metrics? [Asked: Model paper]

REPEATED

Design model metrics measure the quality of the software design before any code is written.

They help catch design flaws early, when fixing them is still cheap.

They measure complexity, so overly complicated designs can be simplified in time.

They measure coupling and cohesion, guiding the designer toward independent, focused modules.

They give an objective score instead of relying only on personal judgement.

They help estimate testing effort, since complex designs need more test cases.

They allow comparison between alternative designs on facts rather than opinions.

Example: if the metrics show one module connected to twenty others, the designer splits it before coding starts.

In short, design model metrics are an early warning system for bad designs.

55 What is software quality? Explain its factors.

IMPORTANT

Software quality means the software meets user needs correctly and reliably.

Correctness: the software does exactly what the requirements ask.

Reliability: it works without failing for a long time.

Usability: it is easy to learn and easy to use.

Efficiency: it uses time and memory wisely without being slow.

Maintainability: it is easy to fix and change in the future.

Portability: it can run on different machines and platforms.

High quality software has few defects and satisfies its users.

Low quality software is full of bugs and frustrates users.

Example: a banking app must be correct and reliable above all else.

In short, quality is not just working software, but software that works well for its users.

56 What are product metrics? Give examples.

IMPORTANT

Product metrics are measurements of the software product itself, not the process.

They help managers judge the quality of the software.

Common examples: the total number of defects found during testing.

Defect density: defects per thousand lines of code, which allows fair comparison.

Example: 5 defects in 1000 lines means a defect density of 5.

Test coverage: the percentage of code or requirements covered by tests.

Higher coverage usually means fewer hidden bugs.

They also include measures like reliability and response time.

Metrics tell whether quality is improving or getting worse over time.

They help decide if the software is ready to release.

In short, product metrics turn quality into numbers that can be tracked.

57 Explain metrics for analysis model, design model, source code, testing and maintenance.

IMPORTANT

Different metrics are used at each stage of the software life cycle.

Analysis model metrics check the requirements model.

They measure size and completeness, like the number of use cases and requirements.

Design model metrics check the quality of the design.

They measure modularity, like coupling between modules and cohesion inside modules.

Source code metrics measure the code itself.

They include lines of code and the complexity of each function.

Testing metrics measure the testing effort and results.

They include defects found, test coverage and defects per test hour.

Maintenance metrics measure how easy the software is to fix.

They include the time taken to fix a defect and the number of change requests.

In short, each phase has its own metrics, and together they track quality from start to finish.

58 Write a short note on code metrics.

IMPORTANT

Code metrics are measurements taken from the source code.

Lines of code is the simplest metric: it counts the size of the program.

It is easy to measure but a long program is not always worse than a short one.

Complexity metrics count the decision points in the code, like ifs and loops.

Higher complexity means the code is harder to understand and test.

Example: a function with 10 if statements is more complex than one with 2.

Defect density measures defects per 1000 lines of code.

It allows fair quality comparison between projects of different sizes.

Example: project A with density 2 is better than project B with density 8.

Code metrics help managers spot risky modules that need more testing.

In short, code metrics turn the code into numbers that guide quality decisions.